ТЕХНИЧЕСКОЕ РУКОВОДСТВО

RGSCenter

Аутентификация, авторизация, ACL, управление учётными записями и аудит команд

Документ описывает фактически реализованный browser-token механизм, выявляет разрыв между аутентификацией и авторизацией и задаёт целевую production-модель доступа для оператора, сервисного инженера и администратора.

Параметр	Значение
Проект	C:\Projects\RGSCenter
Модуль	center / com.rgs.center.auth
Версия документа	1.1
Дата актуализации	21 августа 2026
Статус	Текущее состояние и рекомендуемая целевая архитектура

Содержание

Содержание 2

1. Назначение и границы документа 4

1.1. Текущий уровень готовности . . . 4

1.2. Термины . . . 4

2. Реализованная архитектура аутентификации 4

2.1. Основные компоненты . . . 4

2.2. Последовательность входа . . . 5

2.3. Свойства token-сессии . . . 5

3. Структура базы данных 5

3.1. roles и users . . . 5

3.2. Реализованные authorities . . . 5

4. Текущий разрыв в авторизации 5

4.1. Доступные защищённые маршруты . . . 6

4.2. Риск при подключении реального оборудования . . . 6

5. Целевая модель ACL 6

5.1. Предлагаемые полномочия . . . 6

5.2. Матрица ролей . . . 7

6. Точки принудительной проверки 7

6.1. URL-уровень . . . 7

6.2. Метод-уровень . . . 7

6.3. UI-уровень . . . 7

7. Административный интерфейс 8

7.1. Минимальные функции . . . 8

7.2. Защитные ограничения . . . 8

8. Жизненный цикл пароля и аккаунта 8

8.1. Расширение users . . . 8

8.2. Рекомендуемый процесс . . . 8

9. Аудит действий и команд оборудования 9

9.1. Схема audit_events . . . 9

9.2. Состояния команды . . . 9

9.3. Обязательные события . . . 9

9.4. Что запрещено записывать . . . 9

10. Удаление демонстрационного bootstrap из production **10**

10.1. Целевая схема развёртывания 10

10.2. Проверка запуска 10

11. Управление сессиями **10**

12. Проверки и критерии приёмки **10**

13. Приоритетный план внедрения **11**

14. Файлы реализации **11**

15. Итоговая оценка **11**

1. Назначение и границы документа

Руководство фиксирует текущую реализацию аутентификации RGSCenter и подробно описывает функции, необходимые до production-эксплуатации. Основной вывод: механизм подтверждения личности уже создан, но роль пока не ограничивает доступ к конкретным операциям.

> Текущая система отвечает на вопрос «кто выполнил вход?», но ещё не полностью отвечает на вопрос «что именно этому пользователю разрешено?».

1.1. Текущий уровень готовности

Область	Состояние	Комментарий
Аутентификация	Реализована	BCrypt, серверная token-сессия, HttpOnly cookie, CSRF.
Роли	Частично	ROLE_SERVICE, ROLE_ADMINISTRATOR и ROLE_OPERATOR формируются.
ACL	Не реализованы	Все защищённые endpoint доступны любому аутентифицированному пользователю.
Управление пользователями	Не реализовано	Нет административного UI и lifecycle учётной записи.
Аудит	Не реализован как подсистема	Application log не заменяет неизменяемый журнал действий.
Production bootstrap	Требует исправления	Демонстрационные пользователи и хэши создаются schema.sql.

1.2. Термины

- Аутентификация — проверка личности пользователя.
- Authority / permission — атомарное полномочие, например STATUS_VIEW или DEVICE_CONTROL.
- Роль — набор полномочий, соответствующий рабочей функции.
- ACL — явные правила доступа к endpoint, сервисным методам и объектам.
- Аудит — отдельный журнал значимых действий с пользователем, устройством и результатом.

2. Реализованная архитектура аутентификации

RGSCenter использует stateless browser-token аутентификацию. Spring HTTP session отключена. Случайный токен передаётся браузеру в cookie, а в PostgreSQL сохраняется только SHA-256-хэш токена.

2.1. Основные компоненты

Компонент	Ответственность
SecurityConfiguration	SecurityFilterChain, public routes, stateless policy, CSRF и положение custom filter.
AuthController	GET /login, POST /auth/login, POST /auth/logout, GET /auth/connections.
AuthenticationService	Поиск пользователя, BCrypt-проверка, защита времени ответа для неизвестного login.
AuthSessionService	Выдача, проверка, отзыв и очистка серверных token-сессий.
AuthCookieService	Чтение, установка и очистка RGS_AUTH_TOKEN.
AuthTokenFilter	Восстановление AuthPrincipal и ROLE_* для каждого запроса.
SpaCsrfTokenRequestHandler	Согласование CSRF для HTML-форм и JavaScript-запросов.

2.2. Последовательность входа

Шаг	Участник	Действие
1	Браузер	POST /auth/login: login, password и CSRF token.
2	AuthController	Передаёт данные в AuthenticationService.
3	AuthUserRepository	Выполняет users JOIN roles.
4	PasswordEncoder	Сравнивает пароль с BCrypt-хэшем.
5	AuthSessionService	Создаёт 256-битный token, SHA-256-хэш и запись auth_sessions.
6	AuthCookieService	Возвращает RGS_AUTH_TOKEN как HttpOnly / SameSite=Strict cookie.
7	AuthTokenFilter	На следующих запросах восстанавливает principal и authority ROLE_.

2.3. Свойства token-сессии

- Сырой токен содержит 32 случайных байта и кодируется URL-safe Base64 без padding.
- В базе хранится 64-символьный hex SHA-256, а не bearer token.
- Срок жизни по умолчанию — 8 часов; активность не продлевает expires_at.
- last_seen_at обновляется не чаще одного раза в 15 секунд.
- Для счётчика подключений активным считается браузер, проявлявший активность за последние 90 секунд.
- Logout выставляет revoked_at и удаляет cookie.

3. Структура базы данных

3.1. roles и users

Таблица	Поля	Назначение
roles	id, rolename	Справочник service, administrator, operator.
users	id, login, hash, role	Пользователь, BCrypt-хэш и одна ссылка на роль.
auth_sessions	id, user_id, token_hash, created_at, last_seen_at, expires_at, revoked_at, remote_address, user_agent	Серверные токен-сессии.

Существующая модель допускает ровно одну роль на пользователя. Для текущих трёх функций этого достаточно, однако расширяемая модель полномочий потребует либо role_permissions, либо явного отображения ролей в permission-authorities на уровне Java.

3.2. Реализованные authorities

roles.rolename	Spring Security authority
service	ROLE_SERVICE
administrator	ROLE_ADMINISTRATOR
operator	ROLE_OPERATOR

4. Текущий разрыв в авторизации

AuthTokenFilter корректно добавляет ROLE_* в SecurityContext. Но SecurityConfiguration завершает правила строкой `.anyRequest().authenticated()`. Поэтому роль существует как атрибут principal, но не используется для ограничения endpoint.

> Следствие: operator, service и administrator после входа получают одинаковый серверный доступ ко всем защищённым URL. Скрытие кнопки в HTML не изменит этот факт.

4.1. Доступные защищённые маршруты

Маршрут	Тип	Текущее правило
/, /devices, /devices/status	Просмотр	Любая аутентифицированная роль
/gdrm, /gdrm/status	Просмотр измерений	Любая аутентифицированная роль
/measurement-emulator, /status	Просмотр	Любая аутентифицированная роль
/measurement-emulator/camera/capture	Изменение	Любая аутентифицированная роль
/measurement-emulator/camera/recognize	Изменение	Любая аутентифицированная роль
/measurement-emulator/start	Запуск процесса	Любая аутентифицированная роль
/measurement-emulator/reset	Сброс процесса	Любая аутентифицированная роль
/measurement-emulator/checks/{code}/toggle	Изменение проверки	Любая аутентифицированная роль
/measurement-emulator/leakage/toggle	Изменение проверки	Любая аутентифицированная роль

4.2. Риск при подключении реального оборудования

- Оператор может получить сервисную или диагностическую функцию, если endpoint добавлен без отдельной проверки.
- Администратор учётных записей может случайно получить право опасного физического управления.
- Новый endpoint по умолчанию станет доступен всем вошедшим пользователям.
- Проверка только на странице обходится прямым HTTP-запросом.
- Без аудита нельзя доказать, кто отправил команду и чем она завершилась.

5. Целевая модель ACL

Рекомендуется отделить рабочие роли от атомарных полномочий. Endpoint и сервисные методы должны проверять permission, а не название должности.

5.1. Предлагаемые полномочия

Permission	Разрешаемое действие
STATUS_VIEW	Просмотр витрины, устройств и базовых состояний.
MEASUREMENT_RUN	Запуск штатного автоматического измерения.
MEASUREMENT_CANCEL	Остановка или отмена активного измерения.
CAMERA_CAPTURE	Получение нового кадра камеры.
CAMERA_RECOGNIZE	Распознавание объектов на кадре.
DEVICE_CONTROL	Ручные команды физическим устройствам.
DEVICE_DIAGNOSTICS	Диагностика TCP/COM, протокольные проверки и тестовые команды.
SETTINGS_READ	Просмотр настроек без секретов.
SETTINGS_WRITE	Изменение production-настроек.
AUDIT_READ	Просмотр и экспорт журнала аудита.
USER_ADMIN	Создание, блокировка и изменение ролей пользователей.
SESSION_REVOKE	Завершение активных сессий.

5.2. Матрица ролей

Операция	Permission	Operator	Service	Administrator
Просмотр устройств	STATUS_VIEW	Да	Да	Да
Штатный запуск	MEASUREMENT_RUN	Да	Да	Опционально
Отмена измерения	MEASUREMENT_CANCEL	Да	Да	Опционально
Камера и распознавание	CAMERA_*	Да	Да	Опционально
Ручное управление	DEVICE_CONTROL	Нет	Да	Только отдельно
Диагностика	DEVICE_DIAGNOSTICS	Нет	Да	Опционально
Настройки	SETTINGS_WRITE	Нет	Ограниченно	Да
Пользователи	USER_ADMIN	Нет	Нет	Да
Аудит	AUDIT_READ	Ограниченно	Да	Да
Отзыв сессий	SESSION_REVOKE	Нет	Нет	Да

Администратор системы не обязан автоматически иметь DEVICE_CONTROL: управление пользователями и физическое управление оборудованием относятся к разным зонам ответственности.

6. Точки принудительной проверки

6.1. URL-уровень

Правила requestMatchers дают первую линию защиты и понятный обзор политики доступа.

```
.requestMatchers(HttpMethod.GET, "/devices/**", "/gdrm/**")
    .hasAuthority("STATUS_VIEW")
.requestMatchers(HttpMethod.POST, "/measurement-emulator/start")
    .hasAuthority("MEASUREMENT_RUN")
.requestMatchers("/admin/**")
    .hasAuthority("USER_ADMIN")
```

6.2. Метод-уровень

Для команд устройствам URL-проверки недостаточно. Необходимо включить `@EnableMethodSecurity` и защищать сервисный метод.

```
@PreAuthorize("hasAuthority('DEVICE_CONTROL')")
public CommandResult sendCommand(DeviceCommand command) {
    // Отправка команды устройству
}
```

Это предотвращает обход политики через новый контроллер, scheduler, WebSocket или иной внутренний вызов.

6.3. UI-уровень

- Thymeleaf может скрывать недоступные кнопки через sec:authorize.
- Кнопка должна также иметь понятное пояснение, почему операция недоступна.
- UI-проверка не является механизмом безопасности и никогда не заменяет серверную ACL.
- Для AJAX необходимо корректно обрабатывать 401 и 403 раздельно: 401 — нужен вход, 403 — недостаточно полномочий.

7. Административный интерфейс

Рекомендуемый раздел — /admin/users, доступный только с USER_ADMIN. Он должен работать поверх отдельного сервиса управления учётными записями, а не выполнять SQL из контроллера.

7.1. Минимальные функции

- Список пользователей, роль, статус, время последнего входа и смены пароля.
- Создание пользователя с временным паролем или одноразовым setup-token.
- Блокировка и разблокировка без физического удаления истории.
- Назначение роли или набора permissions.
- Принудительная смена пароля.
- Просмотр активных сессий и отзыв одной либо всех сессий пользователя.
- Фильтрация и просмотр аудита административных действий.

7.2. Защитные ограничения

- Никогда не показывать BCrypt-хэш, token_hash или сырой session token.
- Запретить удаление или блокировку последнего активного администратора.
- Для смены роли, password reset и массового отзыва сессий использовать повторное подтверждение пароля администратора.
- После блокировки пользователя немедленно отозвать все его сессии.
- Все административные изменения фиксировать в audit_events.

8. Жизненный цикл пароля и аккаунта

8.1. Расширение users

Поле	Назначение
enabled	Разрешён ли вход.
must_change_password	Обязательная смена временного пароля.
password_changed_at	Контроль компрометации и политики.
failed_login_count	Счётчик последовательных ошибок.
locked_until	Временная блокировка после подозрительной активности.
created_at / updated_at	История жизненного цикла записи.
last_login_at	Информация для администратора и расследования.
disabled_at / disabled_by	Причина и субъект блокировки.

8.2. Рекомендуемый процесс

Этап	Действие системы
Создание	Администратор задаёт роль; система выдаёт временный пароль или одноразовый setup-token.
Первый вход	Пользователь обязан установить собственный пароль до доступа к приложению.
Обычная работа	BCrypt-проверка, rate limit и журналирование login success/failure.
Смена пароля	Обновляется password_changed_at; все предыдущие auth_sessions отзываются.
Сброс	Короткоживущий одноразовый token хранится только в виде хэша.
Блокировка	enabled=false; все сессии отзываются; новые token не создаются.
Удаление	Предпочтительно логическое отключение, чтобы сохранить аудит.

Минимальная длина пароля — 12–14 символов. Следует запрещать распространённые и скомпрометированные пароли. Плановая частая смена без признаков компрометации обычно ухудшает качество паролей; обязательная смена нужна после reset, утечки или первого входа.

9. Аудит действий и команд оборудования

Application log предназначен для диагностики работы кода. Audit log — для доказуемой истории действий пользователей и оборудования. Эти журналы должны существовать отдельно.

9.1. Схема audit_events

Группа	Поля
Идентификация	id, occurred_at, actor_user_id, session_id, actor_role/permissions snapshot
Источник	remote_address, user_agent
Действие	action, target_type, target_id, device_code
Связь	correlation_id, measurement_id, object_id
Результат	result, error_code, duration_millis
Параметры	sanitized parameters_json без секретов

9.2. Состояния команды

Событие	Смысл
COMMAND_REQUESTED	Пользователь или сценарий запросил действие.
COMMAND_QUEUED	Команда помещена в очередь устройства.
COMMAND_SENT	Байты отправлены в COM/TCP.
COMMAND_ACKNOWLEDGED	Устройство подтвердило приём.
COMMAND_COMPLETED	Физическая операция завершена успешно.
COMMAND_FAILED	Получена ошибка.
COMMAND_TIMED_OUT	Подтверждение или результат не получены вовремя.

COMMAND_SENT нельзя считать успешным завершением физической операции. Для Gamma-PRO, Boson, манипулятора, сканера, SHP и GDRM должны фиксироваться и команда, и подтверждённый результат.

9.3. Обязательные события

- Login success/failure, logout, блокировка и отзыв сессии.
- Создание пользователя, смена роли, reset и смена пароля.
- Изменение настроек и флагов включения устройств.
- Запуск, отмена, аварийная остановка и завершение измерения.
- Ручные команды, включение/выключение высокого напряжения, передача параметров Gamma-PRO.
- Открытие сканирования, результат штрих-кода, команды манипулятору, ошибки и таймауты.

9.4. Что запрещено записывать

- Пароли, временные пароли и password reset token.
- RGS_AUTH_TOKEN, CSRF token и token_hash.
- Полные строки подключения и секреты окружения.
- Неограниченные бинарные кадры протоколов и изображения; сохранять только контролируемые ссылки/хэши при необходимости.

10. Удаление демонстрационного bootstrap из production

Сейчас schema.sql при каждом старте выполняется из-за spring.sql.init.mode=always и через MERGE создаёт service, admin и operator с фиксированными BCrypt-хэшами.

> BCrypt-хэш не раскрывает пароль напрямую, но фиксированный bootstrap опасен: известные или одинаковые данные появляются на всех установках, а аккаунт может остаться с заводским паролем.

10.1. Целевая схема развёртывания

- Перенести DDL и изменения структуры в версионированные Flyway migrations.
- Не создавать пользователей в production migration.
- Оставить demo seed только в dev/test-профилях или test resources.
- Первого администратора создавать одноразовой установочной командой или setup-процедурой.
- Принимать первоначальный секрет извне; не хранить и не печатать его в журнале.
- После bootstrap запрещать повторный запуск setup и требовать сменить временный пароль.
- Для production отключить автоматическое выполнение общего schema.sql после перехода на Flyway.

10.2. Проверка запуска

Production-сборка должна либо иметь заранее созданного администратора, либо переходить в ограниченный одноразовый setup-режим. Она не должна тихо создавать admin/admin, service/service или operator/operator.

11. Управление сессиями

- Администратор видит активные сессии: пользователь, время создания, last_seen, IP и User-Agent.
- Доступен отзыв одной сессии и всех сессий пользователя.
- Смена пароля, блокировка пользователя и изменение критичных прав отзывают активные сессии.
- Проверка token должна учитывать users.enabled; одной проверки expires_at и revoked_at недостаточно.
- Изменение роли уже может применяться при следующем запросе, поскольку principal восстанавливается через JOIN, но отзыв сессии делает изменение явно контролируемым.

12. Проверки и критерии приёмки

Для каждого защищённого действия должны существовать как положительные, так и отрицательные тесты.

Проверка	Ожидаемый результат
Anonymous открывает /devices	Redirect на /login или 401 для API.
Operator открывает /devices	200 OK.
Operator вызывает DEVICE_CONTROL	403 Forbidden.
Service вызывает DEVICE_CONTROL	Команда принята и создан audit event.
Operator открывает /admin/users	403 Forbidden.
Administrator управляет пользователем	Разрешено, действие записано в audit_events.
Заблокированный пользователь использует старый token	401/redirect; сессии отозваны.
Запрос изменения без CSRF	403 Forbidden.
Смена пароля	Старые auth_sessions недействительны.
Production start на пустой БД	Нет автоматически созданных demo accounts.

Критерий готовности ACL: ни один endpoint или сервисный метод, изменяющий состояние процесса, настройки или физическое оборудование, не полагается только на authenticated() или видимость кнопки.

13. Приоритетный план внедрения

Приоритет	Работа	Результат
P0	Утвердить матрицу роль → permission → операция	Однозначная политика доступа.
P0	Защитить команды оборудования и start/reset	Снижен непосредственный производственный риск.
P0	Добавить отрицательные security tests	Защита от регрессий и случайно открытых endpoint.
P1	Добавить enabled и отзыв сессий	Блокировка действительно прекращает доступ.
P1	Убрать demo users из production bootstrap	Нет заводских аккаунтов в production.
P1	Создать audit_events и аудит команд	Прослеживаемость действий и результатов.
P1	Реализовать /admin/users	Управляемый lifecycle аккаунтов.
P2	Flyway и версионированная схема	Безопасное обновление БД.
P2	Политика reset/setup token и re-auth	Завершённый password lifecycle.

14. Файлы реализации

Файл	Назначение / ожидаемое изменение
center/src/main/java/com/rgs/center/auth/SecurityConfiguration.java	URL ACL, method security и обработка 401/403.
center/src/main/java/com/rgs/center/auth/AuthTokenFilter.java	Формирование ROLE_* и будущих permissions.
center/src/main/java/com/rgs/center/auth/AuthSessionService.java	Token lifecycle, отзыв сессий, проверка enabled.
center/src/main/java/com/rgs/center/auth/AuthenticationService.java	Login policy, lockout и обязательная смена пароля.
center/src/main/java/com/rgs/center/auth/AuthController.java	Login/logout/connections; будущая password-flow.
center/src/main/resources/schema.sql	Текущие roles/users/auth_sessions и demo seed; после миграции заменить Flyway.
center/src/main/resources/application.properties	Текущие параметры БД, auth и spring.sql.init.mode.
Новые admin/audit packages	Administrative UI, repositories, services и audit integration.

15. Итоговая оценка

У RGSCenter уже есть хорошая основа: BCrypt, криптографически стойкие token, хранение только token hash, HttpOnly/SameSite cookie, CSRF, ограниченное время жизни и серверный отзыв сессии. Полная замена аутентификации не требуется.

> До production требуется завершить именно слой авторизации и эксплуатации: точные permissions/ACL, административный UI, lifecycle пароля и аккаунта, независимый аудит команд и отказ от демонстрационных пользователей в production bootstrap.

После выполнения P0 и P1 система сможет не только подтверждать личность пользователя, но и доказуемо ограничивать его действия в соответствии с обязанностями и безопасностью физического комплекса.